

Esame OS Internals (Cabodi)

Data: 16/01/2026

QUESTIONS

SECTION 1: THEORETICAL PART

Domanda 1 (Gestione Memoria e Paginazione)

Contesto: Un sistema di calcolo ha le seguenti caratteristiche:

- Dimensione della memoria fisica: 32 GB.
- Spazio di indirizzamento virtuale: indirizzato con 48 bit, indirizzamento al byte.
- Dimensione della pagina: 8 KB.
- Tabelle delle pagine organizzate in modo gerarchico, assumendo che ogni tabella nella gerarchia occupi esattamente una pagina e che ogni Page Table Entry (PTE) abbia dimensione 8 byte.
- TLB con 256 entry.
- Tempo di accesso alla memoria principale (RAM): 100 ns.
- Tempo di lookup della TLB: trascurabile rispetto alla memoria RAM.

A) Calcolare:

1. Il numero di bit dell'offset di pagina.
2. Il numero di bit del numero di pagina virtuale (Virtual Page Number).
3. Quante Page Table Entry sono contenute in una singola tabella delle pagine.
4. Quanti bit del virtual page number possono essere tradotti (al massimo) da una tabella delle pagine a due livelli.
5. Quanti bit del virtual page number possono essere tradotti (al massimo) da una tabella delle pagine a tre livelli.
6. Il numero minimo di livelli della tabella delle pagine necessario per mappare l'intero spazio di indirizzamento virtuale di un processo.

B) Calcoli sui tempi di accesso:

Assumendo che la tabella delle pagine abbia il numero minimo di livelli calcolato al punto A.6. Ricordando che in caso di TLB hit, la traduzione richiede solo il lookup in TLB e un accesso in memoria; in caso di TLB miss, la traduzione richiede un accesso in memoria per ogni livello della tabella delle pagine, più un accesso in memoria per il dato.

1. Calcolare il tempo di accesso in caso di TLB miss (in ns).
2. Determinare il minimo hit ratio della TLB tale che il tempo medio di accesso (EAT) non superi 120 ns.

Domanda 2 (File System e Allocazione Indicizzata)

Contesto: Un file system utilizza un'allocazione indicizzata con due livelli di blocchi indice (indice gerarchico a 2 livelli). Non si tratta di una struttura Unix/UFS basata su inode: la struttura è esattamente la seguente:

- Un blocco indice di primo livello che contiene puntatori a blocchi indice di secondo livello.
- Ogni blocco indice di secondo livello contiene puntatori ai blocchi dati.

I parametri del sistema sono: dimensione blocco su disco = 4 KB; dimensione di un puntatore = 32 bit; partizione = 2 TB (gli altri metadati possono essere trascurati).

A) Calcolare la massima dimensione di un file supportata da questo schema di allocazione. Fornire il risultato in GB.

B) Un file binario ha dimensione 73.011 KB (dove $1 \text{ KB} = 2^{10} \text{ byte}$). Calcolare:

1. Numero di blocchi dati allocati.
2. Numero di blocchi indice di secondo livello allocati.
3. Numero di blocchi indice di primo livello allocati.
4. Frammentazione interna nei blocchi dati (in byte).
5. Frammentazione interna nei blocchi indice (in byte).

C) Si assuma che il file in (B) contenga numeri reali in doppia precisione (64 bit ciascuno). Calcolare:

- Il numero del blocco dati logico (l'indirizzo logico del blocco) che contiene l'elemento con indice $n = 250.000$ (indice degli elementi 0-based).
- Quale blocco indice di secondo livello e quale entry all'interno di esso devono essere acceduti per raggiungere quel blocco dati.

D) Per ciascuna affermazione indicare SI/NO (Vero/Falso):

- Aumentare la dimensione del blocco su disco (mantenendo fissa la dimensione del puntatore) aumenta la dimensione massima del file.
- Con questo schema di allocazione, l'accesso casuale (random access) a un blocco può essere effettuato con un numero $O(1)$ di letture di blocchi indice (cioè un numero costante di blocchi indice).
- Il numero di blocchi indice di secondo livello utilizzati da un file cresce approssimativamente in modo lineare con la dimensione del file.
- I blocchi indice soffrono necessariamente di frammentazione interna anche quando la dimensione del file è un multiplo esatto della dimensione del blocco.

Domanda 3 (Memoria Virtuale e I/O)

Contesto: Si consideri un sistema con memoria virtuale a paginazione a richiesta, MMU con TLB, I/O su file tramite SO, dispositivi I/O con DMA, supporto per memory-mapped files e page replacement gestito dal kernel. Per ciascuna affermazione indicare SI/NO:

- **A)** Usando file mappati in memoria, l'accesso a un byte del file può causare un page fault anche se non viene eseguita alcuna system call di I/O in modo esplicito.
- **B)** Durante un trasferimento DMA verso un buffer utente, le corrispondenti pagine virtuali devono rimanere residenti in memoria fisica per tutta la durata del trasferimento.
- **C)** Se una pagina coinvolta in un trasferimento DMA in corso venisse spostata su disco (swap-out), potrebbe verificarsi corruzione dei dati.
- **D)** La presenza di una TLB garantisce che non siano necessari ulteriori accessi in memoria per tradurre indirizzi virtuali durante operazioni di I/O a blocchi.
- **E)** Aumentare la dimensione della pagina può ridurre il numero di page fault causati da accessi sequenziali a file, ma può aumentare il costo di accessi casuali (random) al file.
- **F)** Quando un file viene letto sequenzialmente usando `read()`, il kernel può sovrapporre (overlap) I/O e calcolo più facilmente rispetto al caso in cui il file sia acceduto tramite memory mapping.
- **G)** Un page fault causato dall'accesso a un file mappato in memoria può bloccare il processo chiamante anche se l'I/O su disco necessario a soddisfare il fault viene effettuato tramite DMA.

SECTION 2: OS161

Domanda 4 (Configurazione OS161)

Contesto: È dato un sistema OS161. Si supponga di aver aggiunto le seguenti istruzioni a `kern/conf/conf.kern`:

```
defoption project
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, con l'aggiunta della riga `options project`. Dire se le affermazioni seguenti sono vere (SI) o false (NO):

- **A)** L'esecuzione di `./config PROJECT`, seguita dal comando `bmake depend`, è sufficiente affinché il file opzionale `syscall/project.c` sia compilato quando si esegue `bmake`.
- **B)** Il file `project.h` viene generato automaticamente dal comando `./config PROJECT`.
- **C)** Il file `opt-project.h` viene generato solo se l'istruzione `options project` compare nel file `PROJECT`.
- **D)** Il file `opt-project.h` contiene la definizione `#define opt_project`.
- **E)** Si supponga di inserire in `main.c` l'istruzione `project_init();`. Per fare in modo che l'istruzione sia compilata solo se l'opzione è abilitata, andranno condizionate le istruzioni come segue:

```
#ifdef OPT_PROJECT
    project_init();
#endif
```

oppure come segue:

```
#if OPT_PROJECT
    project_init();
#endif
```

Domanda 5 (Syscalls e Spinlock OS161)

A) Funzione `sys_getpid`:

Si consideri la funzione `pid_t sys_getpid(void);`. Indicare SI/NO:

1. La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa.
2. La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente.
3. La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall`.
4. La funzione restituisce il pid del processo corrente.
5. La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione).

6. La funzione viene chiamata da `sys_exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione.

B) Funzioni Spinlock:

Si considerino le funzioni OS161 `spinlock_data_get` e `spinlock_data_testandset`. Indicare SI/NO:

1. Per implementarle è necessario utilizzare dei semafori (al fine di risolvere problemi di mutua esclusione).
2. Le funzioni al loro interno chiamano `spinlock_acquire` e `spinlock_release`.
3. Le funzioni sono utilizzate per realizzare `spinlock_acquire`.
4. La funzione `spinlock_release` non chiama la `spinlock_data_testandset` ma la `spinlock_set`, perché non deve gestire una mutua esclusione.

ANSWERS

Domanda 1 (Gestione Memoria e Paginazione)

A) Calcoli sull'Indirizzamento:

1. Bit dell'offset di pagina:

La dimensione della pagina è $8 \text{ KB} = 8192 \text{ Byte} = 2^{13} \text{ Byte}$. Essendo la memoria indirizzabile al byte, l'offset all'interno della pagina necessita esattamente di **13 bit**.

2. Bit del Virtual Page Number (VPN):

L'indirizzo virtuale totale è di 48 bit. Essendo strutturato come $[\text{VPN} \mid \text{Offset}]$, il numero di bit dedicati al VPN si ottiene per sottrazione:

$$\text{VPN} = \text{Indirizzo Totale} - \text{Offset} = 48 - 13 = \mathbf{35 \text{ bit}}.$$

3. Page Table Entry (PTE) in una tabella:

Ogni tabella occupa esattamente una pagina (8 KB) e ogni entry pesa 8 Byte. Il numero di entry è:

$$\text{Numero Entry} = \frac{8192 \text{ B}}{8 \text{ B}} = 1024 = 2^{10} \text{ entry}.$$

Questo significa che ogni singolo livello della tabella delle pagine sfrutta **10 bit** dell'indirizzo per indicizzare le proprie entry.

4. Bit tradotti da una tabella a 2 livelli:

Ogni livello traduce 10 bit. Con 2 livelli si possono tradurre al massimo $10 \times 2 = \mathbf{20 \text{ bit}}$.

5. Bit tradotti da una tabella a 3 livelli:

Con 3 livelli si possono tradurre al massimo $10 \times 3 = \mathbf{30 \text{ bit}}$.

6. Numero minimo di livelli necessari:

Il nostro VPN è composto da 35 bit. Poiché 3 livelli coprono solo 30 bit (insufficienti), è necessario aggiungere un ulteriore livello. Pertanto, il numero minimo di livelli per mappare l'intero spazio di indirizzamento è **4 livelli** (che offrono una capacità di traduzione fino a 40 bit, sufficiente per i nostri 35).

B) Calcoli sui tempi di accesso (EAT - Effective Access Time):

1. Tempo di accesso in caso di TLB miss:

In caso di miss, il tempo di lookup in TLB è nullo (trascurabile). La CPU deve attraversare l'intera Page Table gerarchica in memoria prima di poter accedere al dato reale. Essendoci 4 livelli, occorrono 4 accessi alla RAM per la traduzione. A questi va sommato 1 accesso finale per la lettura/scrittura del dato desiderato.

$$\text{Accessi Totali} = 4 (\text{Page Table}) + 1 (\text{Dato}) = 5 \text{ accessi}.$$

$$\text{Tempo TLB miss} = 5 \times 100 \text{ ns} = \mathbf{500 \text{ ns}}.$$

(Nota: In caso di TLB hit, la traduzione è istantanea e serve 1 solo accesso in RAM per il dato, quindi il tempo di hit è 100 ns).

2. Hit ratio minimo per $EAT \leq 120$ ns:

Sia h l'hit ratio. La formula dell'Effective Access Time è:

$$EAT = h \times T_{\text{hit}} + (1 - h) \times T_{\text{miss}}$$

Sostituendo i valori:

$$120 \geq h \times 100 + (1 - h) \times 500$$

$$120 \geq 100h + 500 - 500h$$

$$120 \geq 500 - 400h$$

$$400h \geq 380 \implies h \geq \frac{380}{400} \implies h \geq 0,95$$

L'hit ratio minimo della TLB deve essere del **95%**.

Domanda 2 (File System e Allocazione Indicizzata)

A) Massima dimensione di un file:

Capacità di un blocco = 4 KB = 4096 Byte.

Dimensione puntatore = 32 bit = 4 Byte.

Puntatori per blocco = $4096/4 = 1024$.

Essendo l'indicizzazione a 2 livelli stretti:

- Il 1° livello contiene 1024 puntatori a blocchi di 2° livello.
- Ogni blocco di 2° livello contiene 1024 puntatori a blocchi dati.
- Numero massimo di blocchi dati = $1024 \times 1024 = 1.048.576$.

La dimensione massima del file si ottiene moltiplicando il numero di blocchi dati per la dimensione del blocco:

$$\text{Max Size} = 1.048.576 \times 4 \text{ KB} = 4.194.304 \text{ KB} = 4096 \text{ MB} = \mathbf{4 \text{ GB}}.$$

B) Calcoli su un file binario da 73.011 KB:

Si ricorda che 1 KB = 1024 Byte.

1. Blocchi dati allocati:

$$\lceil 73.011 \text{ KB} / 4 \text{ KB} \rceil = \lceil 18.252,75 \rceil = \mathbf{18.253 \text{ blocchi dati}}.$$

2. Blocchi indice di 2° livello allocati:

Dovendo mappare 18.253 blocchi dati, e potendone mappare 1024 per ogni blocco indice:

$$\lceil 18.253 / 1024 \rceil = \lceil 17,825 \rceil = \mathbf{18 \text{ blocchi di 2° livello}}.$$

3. Blocchi indice di 1° livello allocati:

Poiché i 18 blocchi di 2° livello richiedono 18 puntatori, e un singolo blocco di 1° livello ne può contenere fino a 1024, è sufficiente **1 blocco di 1° livello**.

4. Frammentazione interna nei blocchi dati:

$$\text{Spazio fisico allocato} = 18.253 \times 4 \text{ KB} = 73.012 \text{ KB}.$$

$$\text{Spazio utile occupato} = 73.011 \text{ KB}.$$

$$\text{Frammentazione dati} = 73.012 - 73.011 = \mathbf{1 \text{ KB (1024 Byte)}}.$$

5. Frammentazione interna nei blocchi indice:

Al 2° livello: 18 blocchi offrono $18 \times 1024 = 18.432$ puntatori. Ne sono utilizzati 18.253.

$$\text{Puntatori liberi} = 18.432 - 18.253 = 179 \implies 179 \times 4 \text{ B} = 716 \text{ Byte}.$$

Al 1° livello: 1 blocco offre 1024 puntatori. Ne sono utilizzati 18.

$$\text{Puntatori liberi} = 1024 - 18 = 1006 \implies 1006 \times 4 \text{ B} = 4024 \text{ Byte}.$$

$$\mathbf{\text{Frammentazione Indici Totale} = 716 + 4024 = 4740 \text{ Byte}}.$$

C) Ricerca dell'elemento 250.000:

Dimensione elemento = 64 bit = 8 Byte.

Poiché l'indice è 0-based, l'elemento $n=250.000$ è preceduto da esattamente 250.000 elementi. Il suo offset in byte all'interno del file è:

$$\text{Offset} = 250.000 \times 8 \text{ Byte} = 2.000.000 \text{ Byte}.$$

Calcoliamo l'indice del blocco logico (da 0) dividendo l'offset per la dimensione del blocco (4096 B):

$$\text{Indice Blocco Dati} = \lfloor 2.000.000 / 4096 \rfloor = \lfloor 488,28 \rfloor = \mathbf{\text{Blocco 488}}.$$

Percorso per raggiungere il blocco 488:

- Indice nel 1° livello (quale blocco di 2° livello): $\lfloor 488 / 1024 \rfloor = 0$. Si accede al **primo blocco indice di 2° livello** (indice 0).
- Entry all'interno del 2° livello: $488 \pmod{1024} = 488$. Si accede alla **489-esima entry** (indice 488).

D) Analisi Affermazioni (Vero/Falso):

- **Aumentare la dimensione del blocco aumenta la dimensione massima: SI.** La formula della dimensione massima per questo sistema è $\text{Size} = (B/4)^2 \times B$. L'aumento del blocco accresce cubicamente la dimensione massima.
- **L'accesso casuale ha costo $O(1)$ in letture indice: SI.** Per accedere a qualsiasi blocco servono sempre e rigorosamente 2 letture di blocchi indice (uno per il 1° livello, uno per il 2°), rendendo il tempo di attraversamento dell'albero costante $O(1)$.
- **I blocchi 2° livello crescono linearmente con il file: SI.** Ogni 1024 blocchi dati allocati (o porzione di essi) viene allocato esattamente un nuovo blocco di 2° livello, creando un rapporto di proporzionalità diretta.
- **I blocchi indice soffrono necessariamente di frammentazione: SI.** Anche se la dimensione del file è un multiplo esatto di 4 KB (annullando la frammentazione dati) e un multiplo esatto di 4 MB (annullando la frammentazione nel 2° livello), ci sarà comunque frammentazione al 1° livello, a meno che il file non occupi l'assoluta dimensione massima del disco riempiendo l'intero albero.

Domanda 3 (Memoria Virtuale e I/O)

Risposte e Motivazioni Teoriche:

- **A) Accesso a mmap causa page fault senza system call? SI.**
Motivazione: Il memory-mapped I/O (mmap) permette di mappare il contenuto di un file direttamente nello spazio di indirizzamento logico del processo. Il processo accede ai dati con normali istruzioni CPU (*load/store*). Se la pagina non è fisicamente in RAM, la MMU hardware solleva automaticamente un page fault, delegando il caricamento al kernel senza alcuna chiamata di I/O esplicita (*read/write*) da parte del processo.
- **B) Le pagine DMA devono rimanere residenti in RAM? SI.**
Motivazione: Durante un trasferimento DMA, il controller hardware scriverà direttamente agli indirizzi fisici prestabiliti bypassando la CPU. Se la pagina venisse sfrattata, il DMA finirebbe per sovrascrivere dati di altri processi o del kernel allocati in quel frame nel frattempo. Le pagine devono essere bloccate (*pinned/locked*) in memoria.

- **C) Swap-out durante DMA causa corruzione? SI.**

Motivazione: Come spiegato al punto precedente, spostare la pagina su disco mentre il controller hardware DMA sta ancora iniettando dati in quel frame fisico provocherebbe una corruzione catastrofica della memoria.

- **D) TLB annulla gli accessi in memoria in I/O a blocchi? NO.**

Motivazione: La TLB è una cache che riduce i tempi medi di traduzione logico/fisica, ma non annulla gli accessi. Gli indirizzi logici dei buffer sorgente/destinazione restano soggetti a traduzione. A fronte di eventuali TLB miss (inevitabili su grossi volumi di I/O), il sistema dovrà comunque accedere in memoria per leggere la Page Table.

- **E) Pagine grandi favoriscono accesso sequenziale ma penalizzano il casuale? SI.**

Motivazione: Una dimensione di pagina grande agisce come un *read-ahead* aggressivo, ottimizzando l'I/O su disco per letture sequenziali. Tuttavia, in caso di accessi casuali, il sistema è costretto a leggere e caricare enormi quantità di dati non necessari dal disco in RAM per soddisfare l'accesso a pochi byte, sprecando banda del disco e inquinando la memoria fisica.

- **F) read() permette sovrapposizione I/O-calcolo più facilmente di mmap? NO.**

Motivazione: Si tratta di astrazioni a livelli diversi. La reale capacità di sovrapporre (overlap) l'operazione di I/O con il calcolo utile sulla CPU dipende dall'utilizzo dei dispositivi hardware DMA e dalle code di implementazione dell'I/O a blocchi nel kernel, a prescindere dal fatto che il processo abbia richiesto i dati tramite system call `read()` o tramite Page Fault da file mappato.

- **G) Page fault da mmap blocca il processo anche se c'è DMA? SI.**

Motivazione: L'eccezione di Page Fault è sincrona rispetto all'esecuzione del thread: l'istruzione assembly corrente non può proseguire senza il dato. Pertanto, il kernel inserisce il processo nella coda dei processi bloccati (*wait queue*). Il DMA svolgerà il trasferimento in background sugli indirizzi fisici, e solo al termine genererà un interrupt che permetterà al kernel di risvegliare il processo.

Domanda 4 (Configurazione OS161)

Analisi delle affermazioni sul sistema di configurazione:

- **A) Sequenza `./config`, `bmake depend`, `bmake` sufficiente per compilare? SI.**

Motivazione: La sequenza indicata è esattamente quella completa per generare l'albero di compilazione, risolvere le dipendenze degli header e compilare il codice sorgente (`project.c`). L'eventuale omissione di `bmake install` è irrilevante ai fini della domanda, poiché tale comando serve unicamente a copiare il kernel già compilato nella directory di root, non partecipando al processo di compilazione stesso.

- **B) Il file `project.h` viene generato automaticamente? NO.**

Motivazione: In OS161, il file di header autogenerato dallo script di configurazione per un'opzione prende sempre il prefisso `opt-`. Pertanto, il file effettivamente generato sarà `opt-project.h`.

- **C) Il file `opt-project.h` viene generato solo se l'istruzione `options project` è presente? NO.**

Motivazione: Avendo dichiarato l'opzione nel file `conf.kern` tramite l'istruzione `defoption project`, il sistema di configurazione genererà *sempre* il file `opt-project.h`. L'assenza

dell'istruzione `options project` nel file di configurazione del kernel non impedisce la generazione del file, ma fa sì che la macro al suo interno venga inizializzata al valore di default 0 (opzione disabilitata).

- **D) Il file contiene la definizione `#define opt_project`? NO.**

Motivazione: Per convenzione del linguaggio C e del sistema di build di OS161, le macro di configurazione autogenerate sono scritte rigorosamente in lettere maiuscole. La definizione corretta inserita nel file sarà `#define OPT_PROJECT`.

- **E) Quale direttiva condizionale utilizzare (`#ifdef` o `#if`)?**

Si deve utilizzare `#if OPT_PROJECT` (la prima opzione è NO, la seconda è SI).

Motivazione: Come illustrato nel punto C, la macro `OPT_PROJECT` viene definita in ogni caso all'interno dell'header (con valore 1 se abilitata, 0 se disabilitata). Di conseguenza, un blocco condizionale basato su `#ifdef OPT_PROJECT` risulterebbe sempre verificato, compilando la funzione `project_init()` anche quando l'opzione non è richiesta. È necessario valutare il *valore* logico della macro tramite la direttiva `#if`.

Domanda 5 (Syscalls e Spinlock OS161)

A) Funzione `sys_getpid`:

1. **Necessaria per `sys_waitpid`? NO.** La funzione `sys_waitpid()` riceve già il PID del processo da attendere come parametro in input, pertanto non ha bisogno di chiamare `sys_getpid()`.
2. **Chiamata direttamente da un processo utente? NO.** Sebbene sia necessaria per implementare la system call `getpid`, trattandosi di una funzione del kernel, non può mai essere invocata direttamente dallo spazio utente a causa della protezione degli indirizzi (privilege isolation).
3. **Chiamata dal kernel, nella funzione `syscall`? SI.** Il processo utente invoca una *trap* (eccezione software). Il controllo passa all'exception handler del kernel, che tramite il dispatcher generale (la funzione `syscall`) invocherà materialmente `sys_getpid`.
4. **Restituisce il pid del processo corrente? SI.**
5. **Chiamata per ottenere il pid di un altro processo? NO.**
6. **Chiamata da `sys_exit` per segnalazioni? NO.** La funzione `sys_exit()` gestisce la terminazione del thread/processo corrente e non ha bisogno di interrogare `getpid` per scoprire il proprio stato, avendo già accesso alle strutture del PCB (Process Control Block).

B) Funzioni Spinlock (`spinlock_data_get` e `spinlock_data_testandset`):

1. **Utilizzano semafori per la mutua esclusione? NO.** Sono primitive atomiche di basso livello fornite dall'hardware. I semafori sono costrutti di alto livello che vengono costruiti *sopra* gli spinlock, non viceversa.
2. **Chiamano al loro interno `spinlock_acquire` e `release`? NO.** Al contrario, sono le istruzioni base atomiche utilizzate per implementare il comportamento della `spinlock_acquire`.
3. **Sono utilizzate per realizzare `spinlock_acquire`? SI.** Permettono di implementare il meccanismo di Test-And-Test-And-Set (TTAS) per acquisire il lock in mutua esclusione.

4. **spinlock_release non usa testandset ma un semplice set? SI.** Quando un thread chiama la release, per definizione possiede già il lock (è già in mutua esclusione). Non ha bisogno di un'operazione *read-modify-write* come la **testandset**; gli è sufficiente una singola scrittura (store) del valore 0, che le architetture garantiscono essere atomica, per liberare la risorsa.